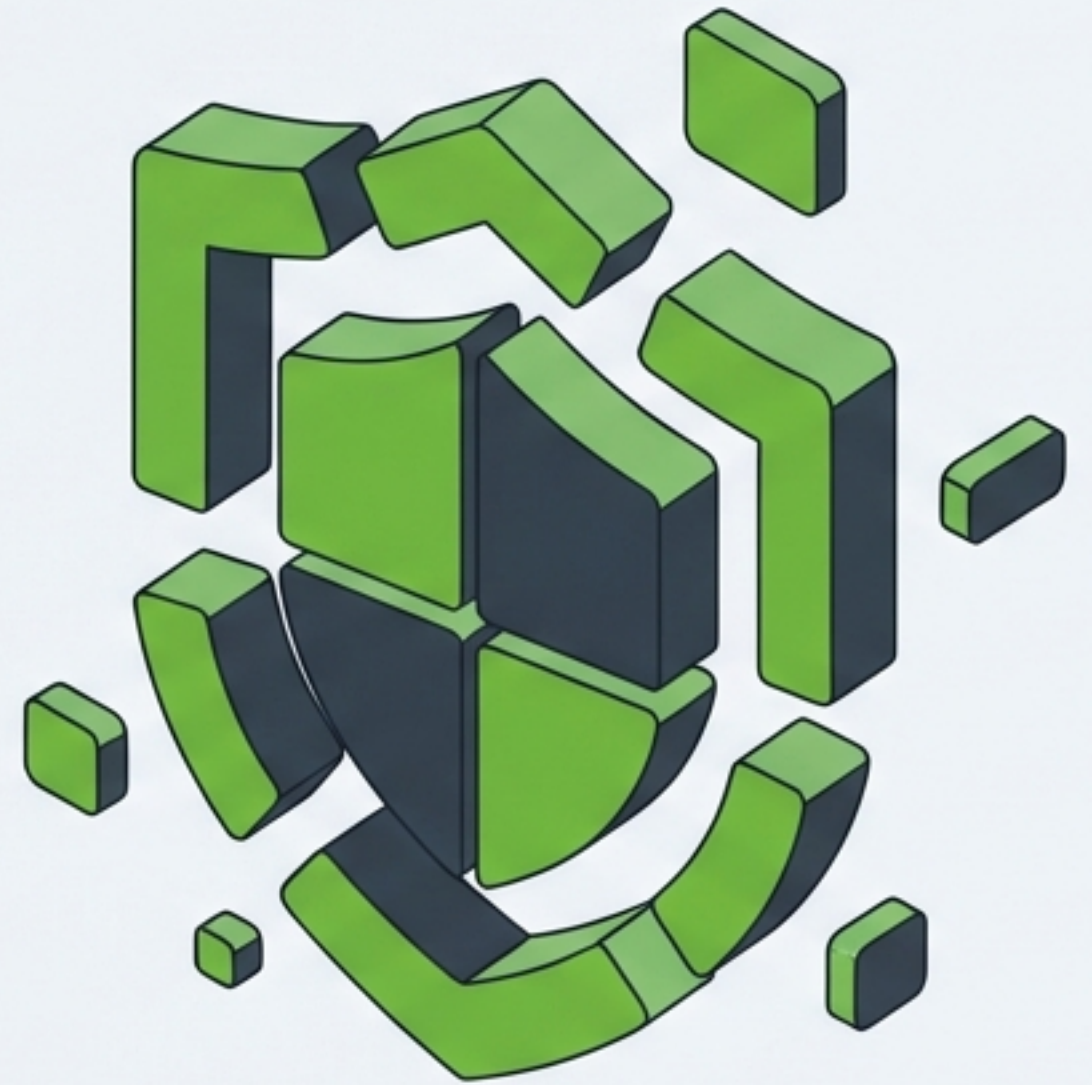


Testing de Microservicios con Spring Boot 4

Estrategias avanzadas de calidad: JUnit
5, Mockito y Testcontainers



Una guía práctica para la arquitectura moderna

El Ecosistema de Testing en Spring

Spring Boot 4 centraliza la configuración de pruebas a través de `spring-boot-starter-test`, eliminando la fricción de configuración manual.

JUnit 5 (Jupiter)

Motor de ejecución estándar

Mockito

Definición de Mocks y Spies

AssertJ

Aserciones fluidas y legibles

Hamcrest

Matchers para validaciones complejas

JSONassert

Verificación de payloads JSON

Insight: A diferencia de la configuración manual, Spring gestiona el ApplicationContext automáticamente, inyectando dependencias tal como lo haría en producción.

Anatomía de un Test en Spring Boot

1

```
@SpringBootTest // Carga el contexto completo  
class MyEngineTest {
```

Bootstraps el contenedor
IoC completo

2

```
@Autowired // Inyección de dependencia real  
Engine engine;
```

Inyecta el bean real
desde el contexto

```
@Test  
void testEngine() {  
    // Given (Preparación)  
    engine.stop();  
  
    // When (Ejecución)  
    engine.start();  
  
    // Then (Verificación)  
    assertThat(engine.isRunning()).isTrue();  
}
```

3

Ciclo AAA: Arrange (Given),
Act (When), Assert (Then)

Estrategias de Dobles de Prueba

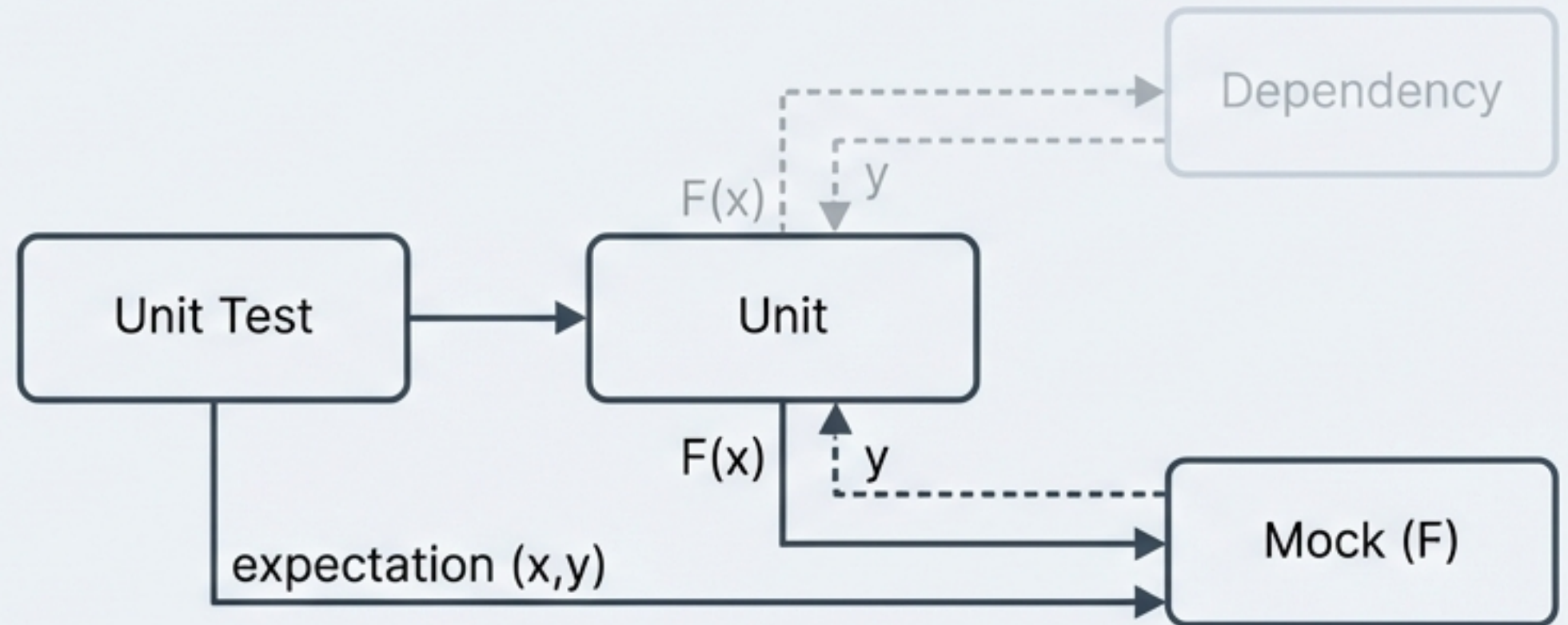
Dummy: Objetos pasados pero nunca usados.

Fake: Implementaciones funcionales simplificadas (ej. DB en memoria).

Stub: Respuestas preprogramadas (estado estático).

Mock: Objetos que verifican comportamiento y expectativas.

Spy: Wrappers sobre objetos reales para rastrear interacciones.



Controlando el Comportamiento con Mockito

Definiendo "Qué hacer cuando..."

```
// Definición del Mock
OrderService service = Mockito.mock(OrderService.class);

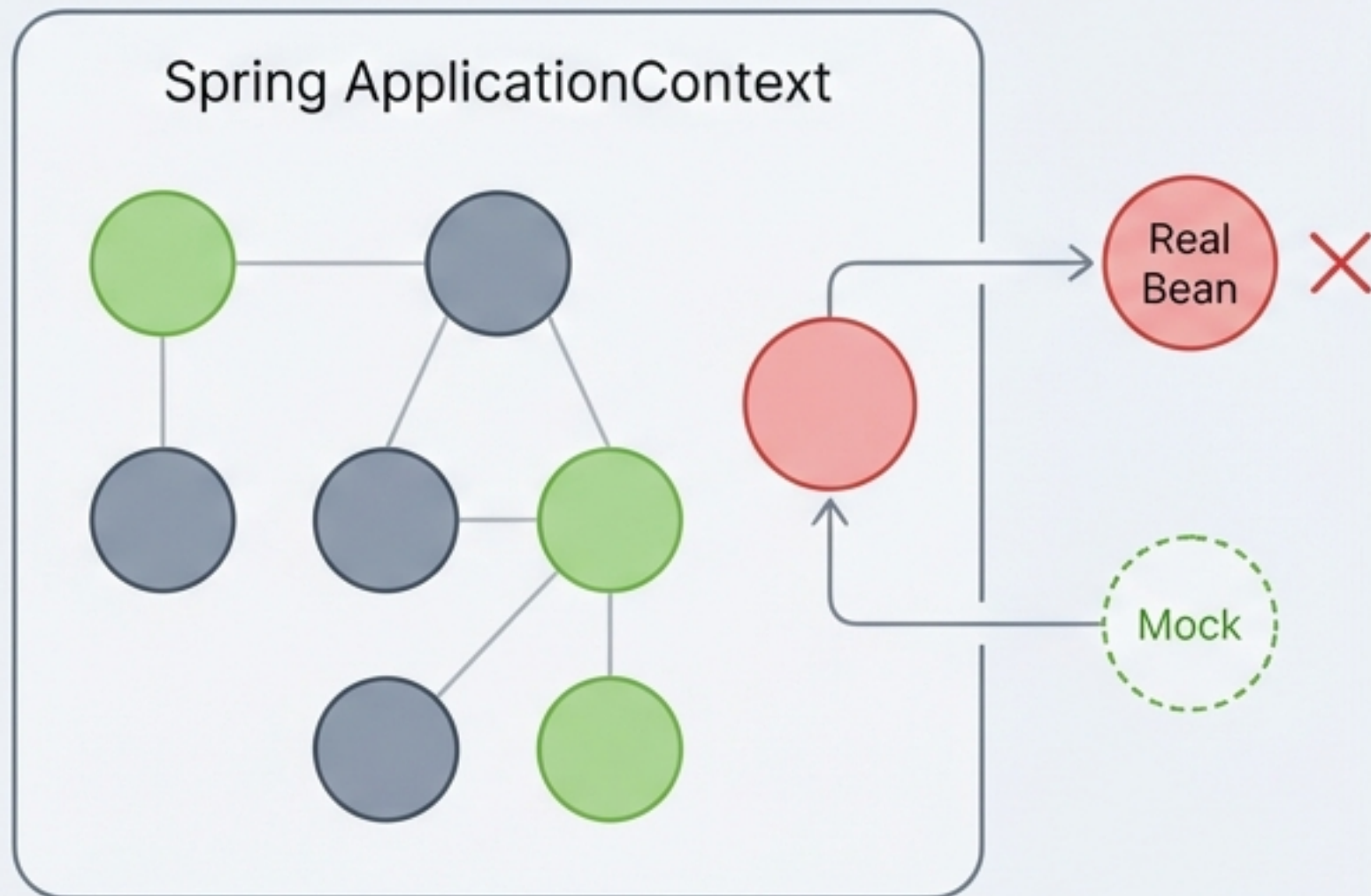
// Stubbing de respuesta exitosa
Mockito.when(service.getOrder(any(UUID.class)))
        .thenReturn(new Order(uuid, "Cliente", 100));

// Stubbing de caso de error (Negative Testing)
Mockito.when(service.getOrder(null))
        .thenThrow (new NullPointerException());
```

Uso de `ArgumentMatchers` (`any()`) para flexibilizar las expectativas.

Simulación de excepciones para probar la resiliencia del código.

Mocks en el Contexto de Spring: @MockBean



```
@SpringBootTest
class OrderFlowTest {

    @MockBean // Reemplaza el Bean real en el contexto 1
    InventoryService inventoryService;

    @Autowired // Inyecta el servicio que usa el mock internamente 2
    OrderProcessor processor;

    @Test
    void processOrder() {
        given(inventoryService.checkStock(any())).willReturn(true);

        processor.execute();

        verify(inventoryService).checkStock(any());
    }
}
```


Espías y Verificación Parcial: @SpyBean

Los espías envuelven una instancia real para rastrear interacciones sin alterar la lógica base.

```
@SpyBean 1
NotificationService notificationService;
```

```
@Test
```

```
void shouldSendEmail() {
```

```
    // Ejecución real
```

```
    orderService.completeOrder();
```

```
    // Verificación de interacción (Side Effect)
```

```
    verify(notificationService, times(1)).sendConfirmation(any());
```

```
    verifyNoMoreInteractions(notificationService);
```

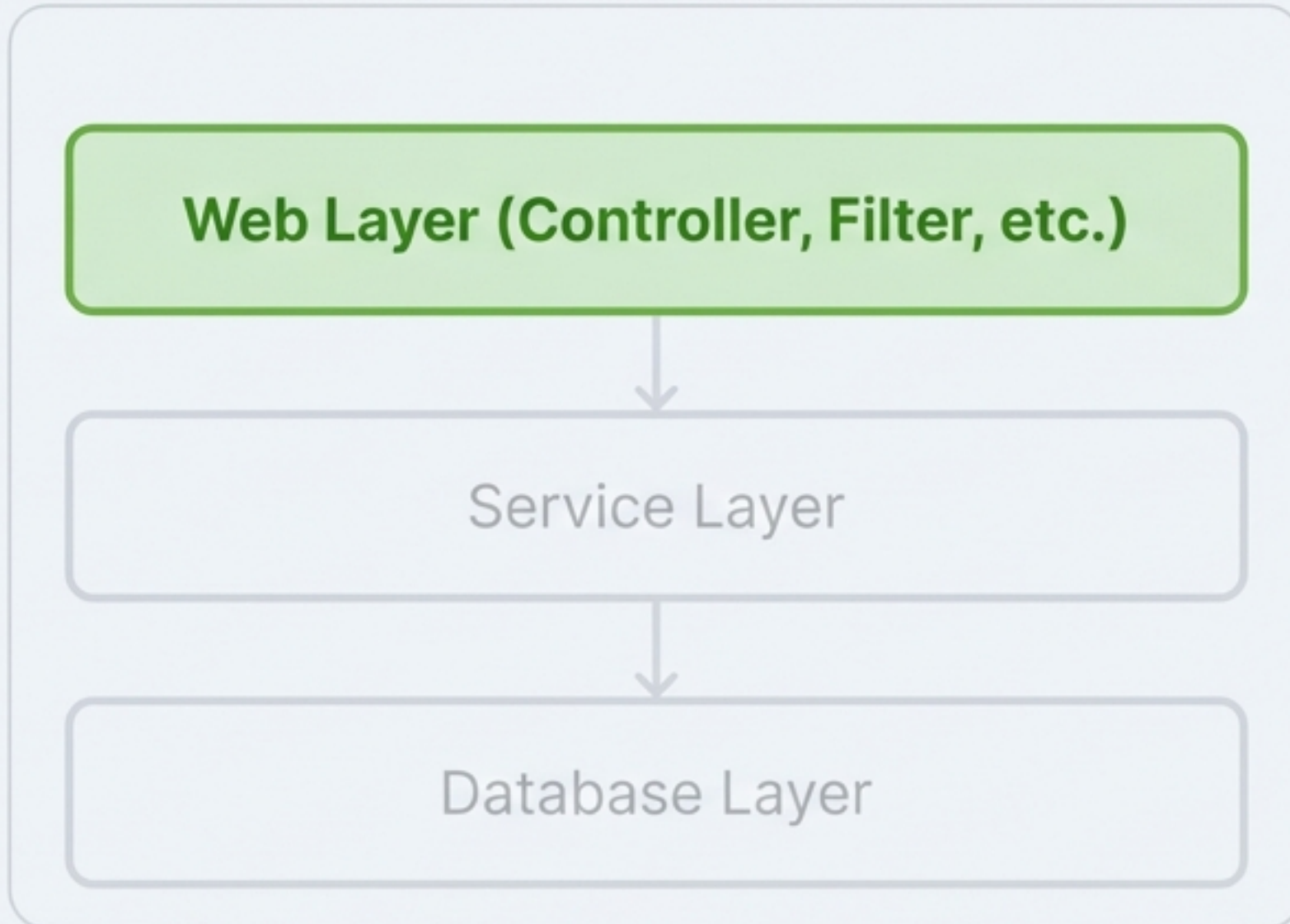
```
}
```

1 Inyecta un espía del Bean, permitiendo verificar interacciones sobre el objeto real.

2 Verifica que el método fue llamado exactamente una vez con cualquier argumento.

Tip: Úsalo con precaución; el exceso de espías puede indicar un acoplamiento fuerte.

Testing de la Capa Web (Slice Testing)



```
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired
    MockMvc mockMvc; // Simula peticiones HTTP sin red

    @MockBean
    OrderService service;

    @Test
    void getOrder() throws Exception {
        mockMvc.perform(get("/orders/{id}", 1))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("SENT"));
    }
}
```


Pruebas Declarativas con REST Assured

Sintaxis fluida BDD para APIs

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
class ApiTest {
    @LocalServerPort
    int port;

    @BeforeEach
    void setup() { RestAssured.port = port; }

    @Test
    void validateEndpoint() {
        given()
            .header("Authorization", "Bearer token")
        .when()
            .get("/api/orders")
        .then()
            .statusCode(200)
            .body("size()", is(5));
    }
}
```


Tests de Integración y Persistencia



En Memoria (H2)

`quarkus.datasource.db-kind=postgresql`

Use `@DataJpaTest`.

Rápido pero arriesgado.

Diferencias de sintaxis SQL

`%test.quarkus.datasource.db-kind=h2`

pueden generar falsos positivos.



Entorno Real (Docker)

`quarkus.datasource.db-kind=postgresql`

Use `@SpringBootTest`.

Realista. Paridad con

producción usando

`%test.quarkus.datasource.db-kind=h2`

contenedores efímeros.

****Insight 2026: Preferir entornos efímeros reales sobre bases de datos en memoria.****

Infraestructura Efímera con Testcontainers

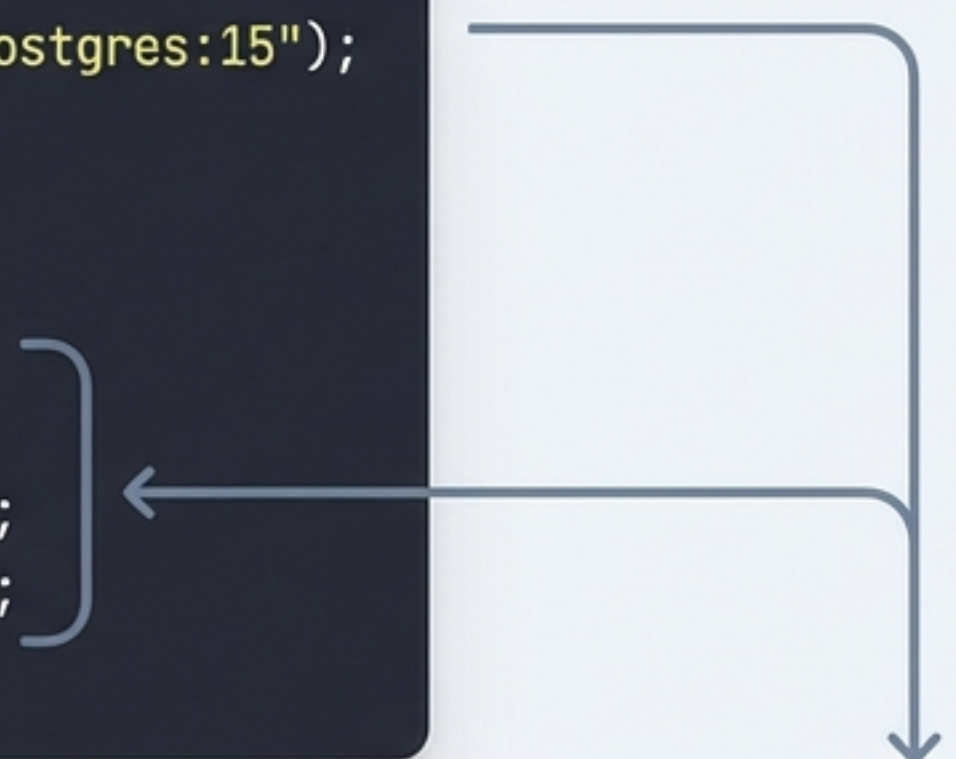


Reemplazamos los mocks de infraestructura por **contenedores Docker desechables** que garantizan paridad con producción.

Conexión Dinámica de Recursos

```
@Container
static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15");

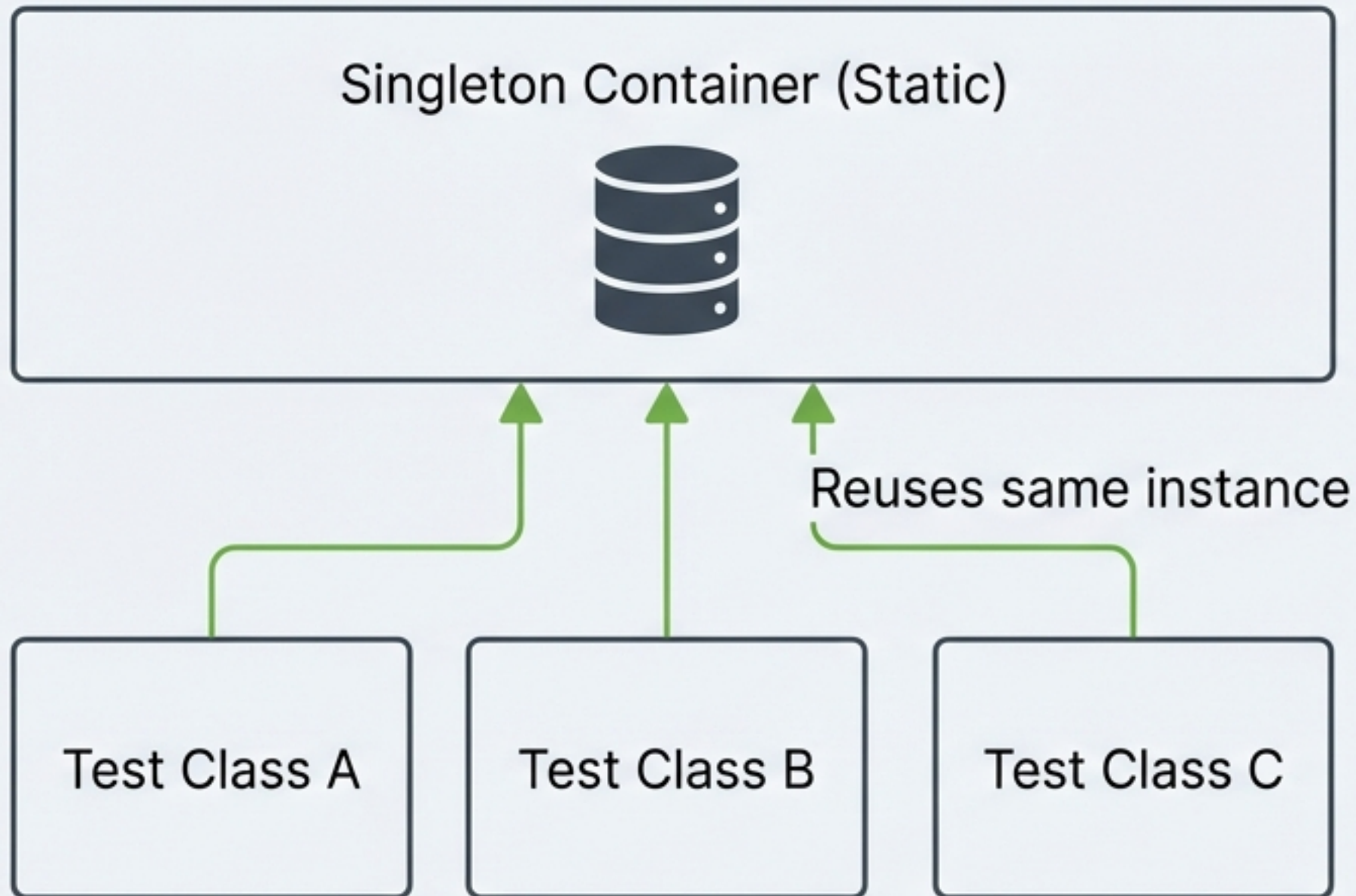
@dynamicPropertySource
static void configureProperties(DynamicPropertyRegistry registry) {
    // Vincula el puerto aleatorio de Docker con la config de Spring
    registry.add("spring.datasource.url", postgres::getJdbcUrl);
    registry.add("spring.datasource.username", postgres::getUsername);
    registry.add("spring.datasource.password", postgres::getPassword);
}
```



Inyección de propiedades en tiempo de ejecución (Runtime)

Vincula el puerto aleatorio de Docker con la config de Spring

Optimización: Contenedores Compartidos



- Declarar el **contenedor** como `static final`.
- Configurar `lifecycle.reuse=true`.
- **Limpiar** datos (TRUNCATE) entre tests.

Testing de Interacciones Asíncronas

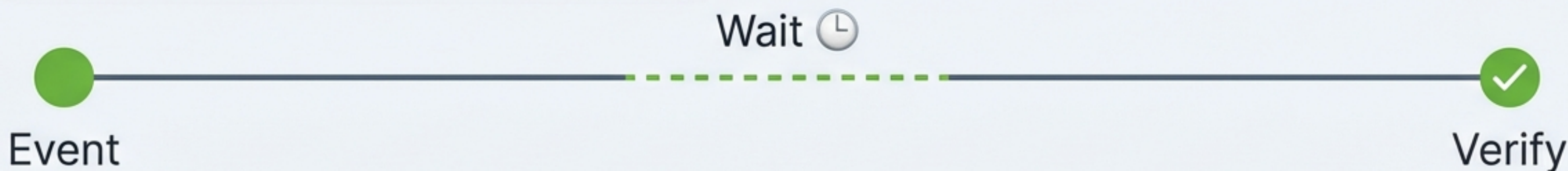
Cuando la respuesta no es inmediata (Eventos, Colas)

Awaitility

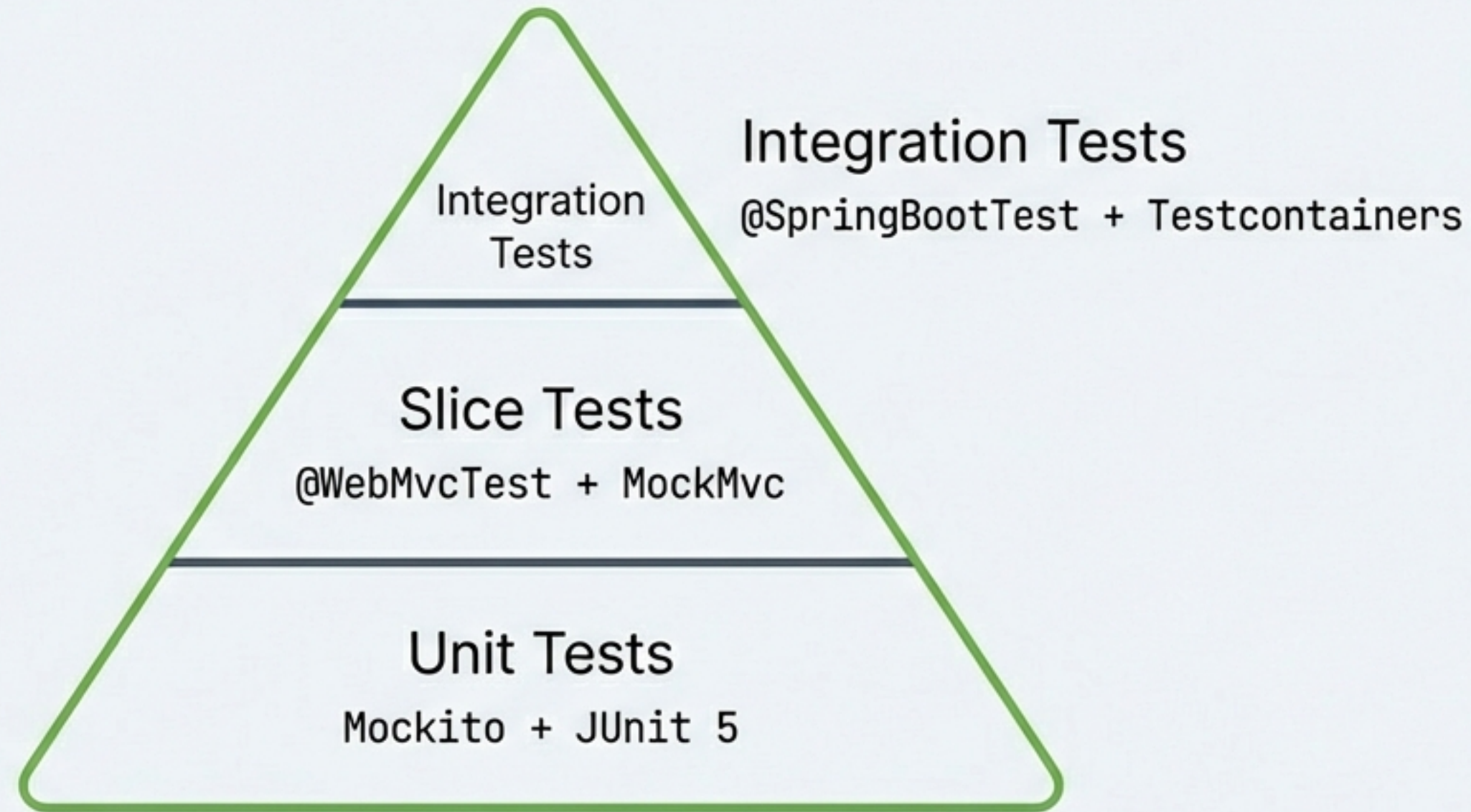
```
await().atMost(5, SECONDS).until(() ->
    repository.count() == 1
);
```

Mockito Timeout

```
verify(kafkaTemplate, timeout(1000)).
    send(any(), any());
```



La Pirámide de Confianza



“La calidad no es un acto, es un hábito automatizado.”